

Code Review — Ștefan · C# Interfaces

Exercițiile ex1–ex5 · verificarea aplicării interfețelor ca **contract** (can-do), separarea capabilităților și principiul open/closed.

Proiect: `stefan-charp-interfaces-starter` · Codul elevului nu a fost modificat — variantele „After” sunt doar în acest document.

Context de rulare. Program.cs invocă doar Interfaces.ex4.Testare (prin using Interfaces.ex4;). ex1 , ex2 , ex3 au clase Testare dar **nu sunt instanțiate** — deci nu rulează. ex5 nu are deloc Testare.cs . La dotnet run se execută practic **numai ex4**.

Structura pe interfețe e corectă peste tot (contracte separate, clase care semnează mai multe contracte, array-uri pe tip de interfață) — scheletul OOP e înțeles. Problemele sunt la **implementarea comportamentului** cerut și la **respectarea constrângerilor**.

● Critice

B1 — ex1 · Boxa.cs:13–16 — validare imposibilă + corp gol.

Condiția `if (value < 0 && value > 100)` nu poate fi niciodată adevărată (un număr nu e simultan `< 0` și `> 100`), iar blocul e gol — deci Boxa nu validează nimic. Trebuie `||` și un `throw` .

B2 — ex2 · CardBancar.cs:21–23 — comisionul e calculat pe sold, nu pe sumă.

Cerința: „la plată se reține suma + comision 1%” (1% din *sumă*). Codul folosește `Sold/100 = 1%` din *sold*. La un sold de 200 și un preț de 10, comisionul iese 2 în loc de 0.1. Afectează și verificarea de fonduri.

B3 — ex2 · Numerar.cs:26–27 și :30–38 — excepția „Not enough cash” e în locul greșit + rambursare inversată.

(1) În `Plateste` , `throw new InvalidOperationException(...)` de pe linia 27 e **cod mort** — vine după `return false` . (2) `Ramburseaza` (unde ar trebui verificarea) **nu verifică sertiarul** și face `SumaDisponibila += pret` . Cerința: „rambursarea **scoate** bani din sertar” → ar trebui `-=` , iar dacă nu sunt destui → `InvalidOperationException("Not enough cash in the drawer")` . Acum excepția cerută nu poate fi declanșată niciodată.

B4 — ex5 · Elev.cs , Produs.cs , Cuvant.cs — miezul exercițiului lipsește.

Cele 3 clase sunt doar containere de date: **nu implementează IComparabil** , nu au `ComparaCu` , nu au `Afisare()` . Fără asta, `Sortator` nu are ce sorta.

B5 — ex5 · Sortator.cs:7–22 + ex5/Testare.cs lipsă — bubble sort incorect și fără rulare.

`Sorteaza` nu compară **vecini**: compară `elemente[i]` cu `elemente[j]` (indici independenți) cu logică `i < j / i > j` care nu sortează corect. Lipsesc și `ex5/Testare.cs` și răspunsul la întrebarea din comentariu.

● Importante

M1 — ex1 · Bec.cs:38–41 — SeteazaIntensitate nu pornește becul.

Cerința: „dacă becul e oprit, SeteazaIntensitate îl pornește automat”. Codul doar setează intensitatea; EPornit rămâne false.

M2 — ex1 · IPornibil.cs:9 / Bec.cs:33 — EstePornit e metodă, nu proprietate.

Contractul cerut: bool EstePornit { get; }. Funcțional merge, dar proprietatea e intenționat parte din lecție („o interfață poate cere și proprietăți”).

M3 — ex1 · IReglabil.cs:7 — SeteazaIntensitateMinima() hardcodează valoarea în fiecare clasă.

Ca să eviți is/as în ModNoapte, ai adăugat o metodă în contract — corect ca idee. Dar hint-ul cerea o **proprietate** Minim (fiecare clasă își declară minimul, telecomanda cheamă SeteazaIntensitate(dispatch.Minim)). Varianta ta duplică logica „ce înseamnă noapte”.

M4 — ex2 · CardBancar.cs:16, Numerar.cs:16, VoucherCadou.cs:15 — validare < 0 în loc de <= 0.

Cerința: „sumă negativă sau zero → ArgumentException”. Cu < 0, o plată de 0 trece nevalidată.

M5 — ex2 · Testare.cs:22 — testul nu declanșează scenariul cerut.

numerar.Ramburseaza(-45.99) aruncă ArgumentException (negativ), nu InvalidOperationException. Lipsește și o rambursare **validă** pe numerar.

M6 — ex3 · Robot.cs:21–34 — predicat cu efect secundar, chemat de două ori.

PoateLivra mută starea (livrare = 0). Dar e chemat o dată de CentruDeLivrari și încă o dată în Livreaza. Funcționează din întâmplare pe scenariul curent, dar orice apel în plus strică numărătoarea.

M7 — ex3 · Testare.cs:26–29 — testul nu acoperă cerința 4.

Doar 4 colete (1, 2.5, 15, 25 kg). Nu se secă autonomia unei drone ca să preia următorul livrator, și nu se atinge „No courier available”.

M8 — ex4 · SmsNotificator.cs:13 — .All(char.IsDigit) este LINQ (interzis).

Constrângerea e explicită: „fără LINQ” și „un for peste caractere și char.IsDigit”. .All(...) e extension method din System.Linq.

M9 — ex4 · MagazinOnline.cs:5,15,47–57 — bonusul Decorator băgat în magazin.

Bonusul cerea o **clasă separată** care împachetează alt INotificator. Aici istoricul e în MagazinOnline printr-un al doilea array + AfiseazaIstoric(), care prindează ToString() -ul obiectelor (numele clasei), nu mesajele. Contrazice ideea „zero modificări în codul magazinului”.

● Cleanups

- C1 — ex1 · Bec.cs:24 — Porneste() setează Intensitate = 100; nespecificat, efect secundar surprinzător.

- **C2 — ex1** · **Priza.cs:24** — `Stare()` returnează doar `"True" / "False"` ; raport sărac.
- **C3 — ex2** · **CardBancar.cs:21** , **Numerar.cs:21** , **VoucherCadou.cs:20** — verificarea `>` strict respinge plata exactă (`sold == sumă`). Probabil vrei `>=` .
- **C4 — ex3** · **CentruDeLivrari.cs:22** — mesajul iese `No courier available for <test4>`; `<...>` era placeholder, nu paranteze literale.
- **C5 — ex3** · **Drona.cs:19** , **Curier.cs:18** , **Robot.cs:28** — limitele cu `<` strict resping coletul fix la capacitate. „Maxim 3” înseamnă `<= 3` .
- **C6 — ex4** · **MagazinOnline.cs:22** — `throw new ArgumentException(nameof(destinatari))` folosește numele parametrului drept *mesaj* — derutant.

Before / After (constatările critice)

B1 — Boxa validare · `ex1/Models/Boxa.cs`

Acum	Corect
<pre>if (value < 0 && value > 100) { } volum = value;</pre>	<pre>if (value < 0 value > 100) { throw new ArgumentException("Volume must be between 0 and 100"); } volum = value;</pre>

B2 — CardBancar comision · `ex2/Models/CardBancar.cs`

Acum	Corect
<pre>if (Sold > pret + Sold/100) { Sold -= pret + Sold/100; return true; }</pre>	<pre>double comision = pret / 100; if (Sold >= pret + comision) { Sold -= pret + comision; return true; }</pre>

B3 — Numerar rambursare · `ex2/Models/Numerar.cs`

Acum	Corect
<pre>// în Plateste: return false; throw new InvalidOperationException(...); // mort // în Ramburseaza: SumaDisponibila += pret;</pre>	<pre>// Plateste: doar return false // Ramburseaza: if (pret <= 0) throw new ArgumentException("Amount must be positive"); if (pret > SumaDisponibila) throw new InvalidOperationException("Not enough cash in the drawer"); SumaDisponibila -= pret;</pre>

B4 — Clasă care semnează contractul · `ex5/Models/Elev.cs`

Acum	Corect
<pre>public class Elev { public string Nume { get; set; } public double Media { get; set; } }</pre>	<pre>public class Elev : IComparabil { public string Nume { get; set; } public double Media { get; set; } public int ComparaCu(IComparabil altul) { Elev alt = (Elev)altul; return alt.Media.CompareTo(Media); // desc } public void Afisare() => Console.WriteLine(Nume + " - " + Media); }</pre>

B5 — Bubble sort pe vecini · `ex5/Models/Sortator.cs`

Acum	Corect
<pre>for (int i = 0; i < n - 1; i++) for (int j = 1; j < n; j++) { int value = elemente[i].ComparaCu(elemente[j]); if (i < j && value > 0) swap(i, j); else if (i > j && value < 0) swap(i, j); }</pre>	<pre>for (int i = 0; i < n - 1; i++) for (int j = 0; j < n - 1 - i; j++) if (elemente[j] .ComparaCu(elemente[j + 1]) > 0) (elemente[j], elemente[j + 1]) = (elemente[j + 1], elemente[j]);</pre>

Ce înveți din asta (mecanismul)

- **Contractul spune „ce trebuie să facă”, nu „aproximativ”.** B1, M1, B3 sunt cazuri unde clasa a semnat interfața dar n-a livrat comportamentul exact. O interfață implementată „pe jumătate” compilează perfect — compilatorul verifică *semnăturile*, nu *sensul*. De-aia contează testul care exercitează fiecare regulă (M5, M7).
- **Un predicat nu mută starea (M6).** `PoateLivra` răspunde la o întrebare; dacă răspunsul *schimbă* lumea, nu mai poți întreba de două ori fără surprize. Separă „pot?” (pur) de „fă” (cu efecte).
- **Constrângerile fac parte din contract (M8).** „Fără LINQ” te forțează să vezi mecanismul (`for + char.IsDigit`) înainte să folosești zahărul care-l ascunde.
- **Decorator = un contract care îmbracă alt contract (M9).** Puterea lui e că magazinul rămâne neatins. Dacă bagi istoricul *în* magazin, ai pierdut exact beneficiul pe care exercițiul voia să-l arate.

Q&A — verificare de înțelegere

1. **B3:** De ce `throw`-ul de pe `Numerar.cs:27` nu se execută niciodată, deși e scris corect sintactic? Unde ar trebui de fapt verificarea „not enough cash” și de ce acolo?
2. **M6:** Dacă cineva ar chema `robot.PoateLivra(x)` de 3 ori la rând ca să „verifice”, fără să livreze, ce s-ar întâmpla cu contorul de reîncărcare? De ce e periculos ca un predicat să aibă efecte secundare?

3. **B4 / ex5:** De ce criteriul de sortare (descrescător după medie) stă în Elev.ComparaCu , și nu în Sortator ? Ce ar trebui să știe Sortator dacă ar cunoaște toate criteriile — și de câte ori l-ai modifica la fiecare tip nou?